Relatório Técnico: Implementação de Pipeline RISC-V

Arthur Carvalho Rodrigues, Débora Luiza de Paula Silva,
Gustavo Henrique Rodrigues de Castro e Mariana Almeida Mendonça

28 de abril de 2026

1 Introdução

Este relatório detalha a implementação de melhorias no processador RISC-V com pipeline de 5 estágios: Busca (IF), Decodificação (ID), Execução (EX), Memória (MEM) e Escrita (WB). O projeto focou na resolução de *Hazards* de controle e dados, visando a integridade da execução matemática e a otimização do desempenho geral do sistema, medido através do índice de Ciclos por Instrução (CPI).

2 Parte 1: Hazard de Controle (BEQ)

2.1 Análise do Problema

Na execução original, o processador falhava ao executar o desvio condicional (*branch*). O erro ocorria porque a Unidade de Branch (`BranchUnit`) não calculava o endereço de destino considerando o *Program Counter* (PC) atual. Consequentemente, o pipeline não realizava o *flush* (limpeza) das instruções carregadas indevidamente. O log de depuração abaixo demonstra que instruções subsequentes que deveriam ser descartadas foram executadas incorretamente, alterando o estado da máquina:

```
FAIL: x5 expected 0 because branch should flush it, got 99
Cycles : 14 | Instructions: 9 | CPI: 1.56
```

2.2 Implementação e Validação

A solução foi implementada modificando a lógica combinacional no arquivo `BranchUnit.v`. O cálculo do endereço alvo (`branch_target`) foi corrigido para realizar a soma do PC no estágio de execução (`pc_ex`) com o valor imediato extraído da instrução (`branch_imm`). Adicionalmente, o sinal `branch_taken` foi ativado quando a condição de igualdade dos registradores foi satisfeita.

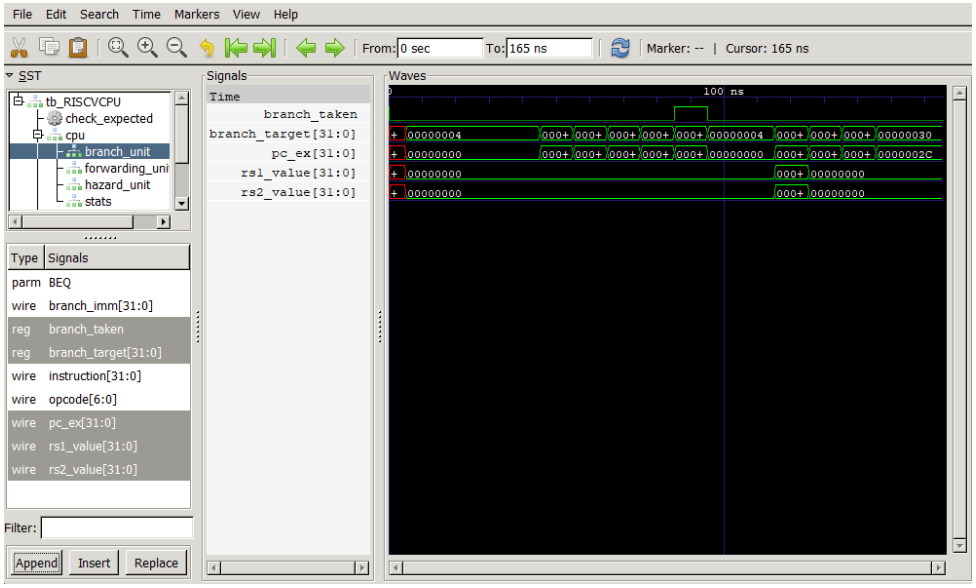

Figura 1: GTKWave demonstrando o sinal branch_taken e o cálculo do PC de destino corrigidos.

Após a modificação, a simulação confirmou o funcionamento do desvio, resultando na limpeza bem-sucedida de uma instrução indevida do pipeline (1 *flush*):

```
Cycles : 14 | Instructions : 7 | Branches : 1 | Flushes : 1 | CPI : 2.00
```

3 Parte 2: Resolução via Software (NOPs)

Durante a execução de instruções sequenciais dependentes, identificamos conflitos do tipo *Read After Write* (RAW). Como a arquitetura inicial não possuía uma unidade de *forwarding* para adiantar os resultados, foi necessário intervir via software, inserindo instruções "bolha"(`NOP`) manualmente no código Assembly.

3.1 Atraso de Escrita no Banco de Registradores

Durante a análise de temporização, identificou-se uma característica intrínseca do simulador *Icarus Verilog* utilizado: o sistema não realiza a separação clássica de leitura e escrita no estágio de *Write Back* (WB) dentro de metades opostas do mesmo ciclo de *clock*.

Devido a essa limitação síncrona (onde leituras e escritas ocorrem simultaneamente na borda de subida), o dado escrito no ciclo 5, por exemplo, só se torna estável e disponível para leitura no ciclo 6. Por este motivo, a implementação de apenas 2 NOPs demonstrou-se insuficiente, forçando a adoção de **3 NOPs** entre cada par de instruções dependentes para garantir a coerência dos dados lidos no estágio ID.

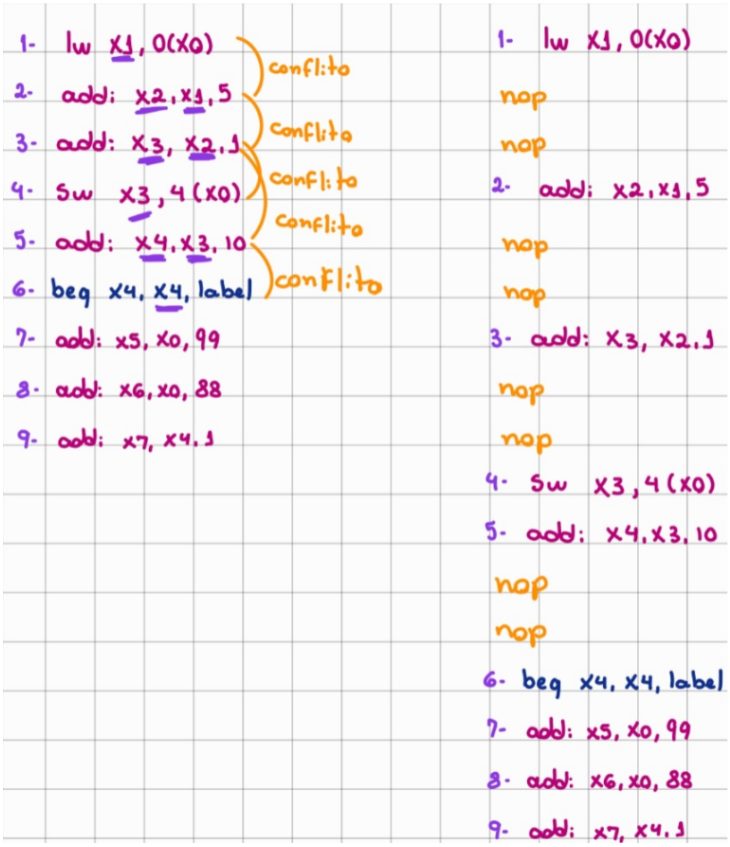

Figura 2: Mapeamento de bolhas manuais para sincronização de dados considerando a latência do simulador.

A mitigação via software resolveu as inconsistências matemáticas, porém degradou severamente a eficiência do pipeline:

```
--- LOG DE EXECUÇÃO COM NOPS ---
Cycles : 26 | Instructions: 7 | CPI: 3.71
PASS: all expected results match.
```

4 Parte 3: Resolução via Hardware

Para restaurar a eficiência do processador, o código Assembly foi retornado ao seu estado original (sem `NOPs`), transferindo a responsabilidade de resolução de conflitos para a própria arquitetura de hardware.

4.1 Forwarding Unit (Adiantamento)

Foi implementada a lógica de *Bypassing* na `ForwardingUnit`. A unidade monitora os registradores fonte no estágio de Execução (`idex_rs1` e `idex_rs2`). Caso esses registradores coincidam com os registradores de destino de instruções que estão nos estágios de Memória (`exmem_rd`) ou *Write Back* (`memwb_rd`) — garantindo que não seja o registrador nulo `x0` —, multiplexadores redirecionam os dados recém-calculados diretamente para a entrada da ALU. Implementou-se também uma lógica de prioridade estruturada via `if/else if` para conflitos duplos, garantindo que a instrução mais recente (no estágio MEM) sempre tenha prevalência sobre a mais antiga (no estágio WB).

4.2 Hazard Detection Unit (Stalls)

Para o *Hazard* específico de carga (*Load-Use*), o adiantamento não é suficiente, visto que o dado provém da memória principal e possui maior latência. A `HazardDetectionUnit` foi desenvolvida para detectar se a instrução no estágio de Memória é um Load (`exmem_op == LW`) e se o seu destino coincide com as fontes da instrução atualmente em Execução. Nesse cenário isolado, o hardware injeta automaticamente uma bolha de atraso (*stall* = 1).

4.3 Comparativo de Desempenho Final

A transição da solução em software para hardware resultou em um sistema funcionalmente correto e altamente otimizado.

Tabela 1: Comparativo Geral de Métricas do Pipeline

Métrica	**Solução Software (NOPs)**	**Solução Hardware (Final)**
Ciclos Totais	26	**15**
Instruções Úteis	7	7
Bypasses Utilizados	0	**6**
Stalls Automáticos	0	**1**
CPI Final	**3.71**	**2.14**

A execução final dispensou a ociosidade excessiva de ciclos de *clock*, reduzindo o tempo de processamento em aproximadamente 42%.

```
--- LOG FINAL DE HARDWARE ---
Cycles : 15 | Instructions: 7 | Stalls : 1 | Bypasses : 6 | CPI : 2.14
PASS: all expected results match.
```

A Dificuldades Encontradas

Durante o desenvolvimento, a equipe encontrou desafios na configuração inicial do ambiente Icarus Verilog no sistema operacional Windows. Além disso, o comportamento atípico do simulador em relação à temporização do *Write Back* (que não divide o ciclo em metades) exigiu um esforço extra de *debugging* para identificar a necessidade de injetar 3 NOPs ao invés dos 2 previstos na literatura teórica. Por fim, houve um desafio na compreensão da persistência de `NOPs` ocultos na memória ao transitar da Parte 2 para a Parte 3, que mascarava a ativação correta do Forwarding no log de estatísticas.

B Uso de Inteligência Artificial e Transcrição de Prompts

O modelo de Inteligência Artificial Gemini (versão Pro 3.1) foi utilizado como ferramenta de tutoria. A IA auxiliou na análise estrutural do tempo do pipeline (visualização de ciclos), depuração de erros de sintaxe em Verilog e refinamento da lógica booleana dos arquivos de hardware.

Nota sobre o compartilhamento de chat: O link direto para o histórico da conversa com a IA não pôde ser gerado pois as diretrizes de segurança da interface desabilitam

a criação de URLs públicas quando imagens e arquivos locais (como *prints* de código e PDFs) são carregados como contexto do usuário. Portanto, as principais interações estão transcritas abaixo. Caso o professor considere necessário analisar a íntegra da conversa para atestar a originalidade e o processo de tutoria, nos colocamos à total disposição para fornecer a exportação completa do chat em registro de vídeo da interação na plataforma.

Histórico de Prompts Relevantes

Para documentar a evolução lógica do projeto, listamos abaixo as principais interações e questionamentos submetidos à IA que guiaram as decisões de arquitetura e depuração do código Verilog:

- **Início da Parte 2 (Mapeamento de Hazards RAW):** *"perfeito, bora pra explicação da parte 2 então! (...) Instrução 1 IF ID EX MEM WB... quando a instrução 2 pega o valor x1, a instrução 1 ainda não completou seus ciclos (WB)... seria esse o problema, correto?"*

- **Validação Visual do Pipeline:** *"[Envio de imagem do caderno contendo a primeira estrutura de bolhas] Seria algo assim?"* seguido pelo refinamento estrutural: *"[Envio de segunda imagem otimizada] Assim seria então?"*

- **Descoberta da Latência do Icarus Verilog:** *"após essas modificações, esse foi o resultado de vvp simv... FAIL: x2 expected 15, got 5"* (Prompt crucial que revelou a necessidade de alterar a arquitetura de 2 NOPs para 3 NOPs devido ao comportamento não-bloqueante na borda de subida do *clock*).

- **Conceituação do Forwarding:** *"gostaria de pensar na lógica do forward primeiro... A função do Forwarding é pegar um dado que acabou de ser calculado mas que ainda não foi gravado no registrador e adiantar para a entrada da ALU, certo?"*

- **Discussão sobre Direção e Casos de Uso:** *"então se uma instrução 1 estiver no estágio 4 e a instrução 2 no estágio 3 podemos fazer forward? tipo: add x13, x11, x12 | lw x13 0(x0) ?"* (Questionamento que validou a regra de que o *forwarding* só atende leituras, e não escritas de destino do *Load*).

- **Prioridade Temporal (Conflitos Duplos):** *"no primeiro if checamos se é EX -> MEM e no else if se é EX -> WB?"* (Confirmação da implementação estrutural de prioridade para capturar o dado mais recente).

- **Mapeamento Arquitetural (Hazard Detection):** *"de onde vem IFIDrs1 e IFIDrs2 ?"* e a posterior depuração do arquivo do professor: *"porém ifid_rs2 não está no código"* (Interações que ajustaram a teoria clássica de detecção de *Stalls* para a arquitetura específica fornecida).

- **Depuração do Testbench (NOPs Fantasmas):** *"[Envio do log de testes com PASS, mas com CPI 3.71, 0 Stalls e 0 Bypasses] aqui estão os testes comentando os nops e com as outras modificações"* (Este prompt levou à descoberta de que a função `clear_memories()` do *testbench* estava preenchendo os espaços dos índices comentados com códigos hexadecimais de `NOP`, impedindo o acionamento do *Forwarding*).